

Assignment 2: Distributed Banking Platform

Due date: _____1405/08/15_____ Submission: ____ LMS,Github,Dockerhub _____

Assignment Reviewers: Alireza Bagherzadeh | Mehdi Shekari

Sharding, Replication & Distributed Transactions

1. Goal

Accounts are partitioned across 2 independent shards, each shard has multiple application replicas, and cross-shard transfers are coordinated with Two-Phase Commit (2PC). Everything runs with Docker Compose on a single host.

2. Components

- Banking Client: balance inquiry, deposit, withdraw, transfer. Talks only to the Gateway and never needs to know shards or replicas.
- Shard-Aware Gateway: (1) shard selection by account ownership, then (2) replica selection among healthy replicas of that shard (Round Robin, Least Connections or similar, documented). Invokes the Coordinator for cross-shard transfers.
- Banking Shards: 2 shards, each with its own persistent database and at least 2 application replicas sharing that shard database. Replicas hold no independent authoritative balances.
- Transaction Coordinator: assigns a unique transaction ID, runs prepare/commit, durably records decisions, recovers unresolved transactions. May share a container with the Gateway if responsibilities are logically separate.
- Shard Map: describes which shard owns which account range; stored in a config file or metadata database and used by the Gateway for routing.

Shard	Account range	Components
Shard A	A0001–A1000	Bank Server A1, A2, Database A
Shard B	A1001–A2000	Bank Server B1, B2, Database B

3. Sharding and Routing

- Partitioning may be range-based, hash-based or another documented deterministic method. Each account has exactly one authoritative shard; the full dataset must not be copied to every shard.
- Requests for an account always go to its owning shard, then are spread across that shard's replicas. Never spread an account's requests to unrelated shards for global balance.
- Replica Consistency: Concurrent operations across replicas of the same shard must not cause lost updates or inconsistent balances. Implement and explain an appropriate concurrency control mechanism.
- Availability: the Gateway detects unavailable replicas (e.g. health checks) and routes new requests to a healthy replica of the same shard. Database replication and failover are not required.

4. Banking Operations

- Local operations (balance, deposit, withdraw): routed to the owning shard; Assignment 1 ACID and durability requirements still apply.
- Same-shard transfer: one atomic local database transaction; both accounts update or neither does.
- Cross-shard transfer: needs 2PC. Example: A0500 (Shard A, 1000) sends 300 to A1500 (Shard B, 500) → 700 and 800. Never debit or credit only one side permanently.

5. Distributed Transactions (2PC)

Protocol

- Phase 1 – Prepare: Coordinator identifies shards and sends PREPARE; each participant validates, durably prepares (without committing) and votes.
- Phase 2 – Commit/Abort: COMMIT only if all vote YES, otherwise ABORT. The decision is durably recorded before it is sent. Participants must handle repeated COMMIT/ABORT safely.
- Two unrelated sequential updates are not a valid implementation. PostgreSQL prepared transactions are suggested; other storage must show equivalent guarantees.

Durable state

Each transaction has a unique ID and persistent status: INITIATED, PREPARING, PREPARED, COMMIT_DECIDED, ABORT_DECIDED, COMPLETED (more may be added). The Coordinator must rebuild unresolved transactions after a restart.

Do not report success before all participants confirm COMMIT. Unresolved transactions must expose a queryable PENDING/UNKNOWN status by transaction ID.

Recovery scenarios

Scenario	Situation	Required behavior
A	One shard votes YES, another rejects	Safely abort all prepared participants
B	All prepared, Coordinator stops	After restart, recover durable state and continue or resolve
C	COMMIT recorded, not all acknowledged	Complete the recorded COMMIT; never replace it with ABORT because a participant is temporarily down

6. Required Experiments

#	Experiment	What to show
1	Shard distribution	Test accounts across both shards are stored in the assigned databases; Gateway routes correctly
2	Replica load distribution	Many requests to one shard's accounts are handled by multiple replicas; logs identify the replica
3	Concurrent replica operations	Concurrent withdrawals/deposits through different replicas of one shard give correct balances
4	Replica interruption	Stop one replica under load; Gateway stops selecting it and uses another healthy replica
5	Local and cross-shard transfers	Show same-shard and cross-shard transfers; explain which need coordination and why
6	Interrupted distributed transaction	Stop Coordinator after durable COMMIT but before all acks; restart and show completion
7	Concurrent cross-shard transfers	Concurrent transfers incl. overlapping accounts; verify balances, states and conservation of money

7. Expert Bonus (+12%): Live Re-Sharding Under Adversarial Failures

Add Shard C (its own persistent database and at least two application replicas) at runtime and move A1501–A2000 from Shard B to C without stopping the banking platform. Unlike a basic migration, the migrating accounts must continue accepting concurrent operations during bulk copy and catch-up. A static restart, full-range freeze throughout copying, manual database repair, or two writable owners does not qualify.

- **Online copy + change capture (3 pts):** take a consistent snapshot, then capture and replay ordered changes made while the snapshot is copied. Use durable change positions and idempotent replay; do not rely on a one-time copy or unbounded in-memory queues. Other accounts must remain available; migrating accounts may be briefly paused only for final cutover.
- **Atomic ownership + stale-route fencing (3 pts):** use a persistent, monotonically increasing Shard Map epoch. Fence writes on the old shard before activating the new owner; enforce epochs at the shard write path, not only in the Gateway. Deliberately send stale requests after cutover and prove that none can modify the old owner.
- **2PC and exactly-once effects (3 pts):** continue same-shard and cross-shard transfers during migration. Resolve or safely block in-flight prepared transactions before ownership changes; never discard a durable COMMIT decision. Require client request IDs, safe retries, duplicate-message tolerance and a transaction-status query for uncertain outcomes.
- **Crash-consistent state machine (3 pts):** persist migration phase, checkpoints, epoch, and cutover decision. Recover automatically after abrupt container termination during (i) snapshot, (ii) delta replay, (iii) fencing, and (iv) after map activation but before cleanup. Recovery must converge without data loss or manual edits; old data is removed only after verified completion.

8. Expert Bonus II (+10%): Autonomous Self-Healing

Implement a custom Self-Healing Controller that automatically detects failed or unresponsive banking replicas, creates replacement containers, and registers them with the Gateway without interrupting healthy replicas.

Requirements:

- Maintain the configured number of healthy replicas automatically.
- Detect failures and create new replacement containers.
- Recover from controller crashes without creating duplicate replicas.
- Preserve transaction correctness during failures and recovery.
- Demonstrate automatic recovery under concurrent banking requests and injected crashes.

Restriction: Kubernetes, Docker Swarm, prebuilt self-healing tools, and Docker automatic restart policies are prohibited. Students must implement the recovery logic themselves.

9. Deliverables

- **Source code:** Upload the complete project to a GitHub repository.
- **Docker deployment:** Push all required Docker images to Docker Hub.
- **Technical report (max 10 A4 pages):** architecture, sharding and Shard Map, load balancing, concurrency control, 2PC protocol with sequence diagram, durable decisions and recovery, limitations and results (plus migration protocol for bonus I and self-healing mechanism for Bonus II).
- **Demo video (max 15 minutes):** Compose deployment, both shard databases, replicas handling requests, shard-aware routing, local and cross-shard transfers, Coordinator interruption and recovery, concurrent tests (plus shard expansion and migration for Bonus I and automatic replica recovery for Bonus II).